Revisiting the Average Case Complexity of Multilevel Syllogistic

Lars Warren Ericson[1]

[1]Catskills Research Company
[1]lars.ericson@catskillsresearch.com

September 4, 2026

ORCID: 0000-0001-8299-9361
Primary Category: cs.LO (Logic in Computer Science)
Secondary Category: cs.CC (Computational Complexity)
Lean 4 formalization: https://github.com/catskillsresearch/avg_case_mls

Abstract

We describe a Lean 4 formalization revisiting NYU Courant Technical Report TR1995-711 on the average-case complexity of Multilevel Syllogistic (MLS). The development encodes Reischuk–Schindelhauer average-case classes, an axiomatic MLS/EMLS semantics layer, a partial Ferro–Omodeo–Schwartz decision procedure with proved soundness and partial completeness on a membership-free fragment, serialization and step budgets, and conditional NP-average completeness and non-AvP hardness corollaries modulo explicitly documented structural axioms. Full Lean sources are inlined in the appendix modules.

1 Abstract

We revisit Cox, Ericson, and Mishra's 1995 Courant technical report TR1995-711 (*The average case complexity of multilevel syllogistic*) in Lean 4. The report's combinatorial core — the Chvátal–Szemerédi resolution lower bound and its averaged corollary — formalizes completely: 6,833 lines across eighteen modules, no `sorry`, no hypothesis packages. So do the three SAT-to-fragment reductions (Theorems 5.1–5.3), Example 4.1 with its explicit normalizing constant, and Theorem 4.4 on the timed layer when honest invertible reductions are taken seriously.

Against that positive record, three independent defects in this repository's initial untimed adaptation of the Reischuk–Schindelhauer vocabulary make several encoded theorems carry no complexity content. These are defects of the Lean adaptation: RS93 and TR1995 tie running time and NP membership to machines, conditions the adaptation omitted. We prove the resulting collapses constructively, supply repairs, and re-prove Theorem 4.4 against strengthened definitions. Every displayed proof has a runnable Lean counterpart in `Exposition/`.

---

2 Introduction

In the Correct Program Technology (CPT) vision of the 1970s–80s, programmers would write code alongside mathematical specifications, and a compiler integrated with an automated theorem prover would verify conformance [DS77]. Decision procedures for decidable sublanguages

of set theory — Multilevel Syllogistic (MLS), Elementary MLS (EMLS), and related fragments — were central to that program [FOS80, Sny90a]. But MLS and EMLS are NP-complete in the worst case, and extensions with Presburger arithmetic are much worse. Goldberg's early experiments [Gol79] suggested that resolution-based SAT solvers might nonetheless perform well on *average* inputs, motivating a formal theory of average-case complexity [BDCGL89, Lev86, RS93, Gur91].

TR1995-711 [CEM95] applies that theory to MLS satisfiability and related verification problems. It states theorems, not conjectures, for resolution lower bounds (Section 3), average-case completeness and transfer (Section 4), and polynomial-time reductions from SAT to MLS, EMLS, and FP/LP (Section 5).

This note asks a narrower question: **what survives contact with a proof assistant?** We formalized the report's definitions and theorems in Lean 4 against Mathlib. The answer splits cleanly. The combinatorial and syntactic content is real and now verified. Several complexity-theoretic statements collapse because the repository's initial untimed encoding omits machine-time conditions and admits zero-mass witnesses.

Our contribution is therefore twofold:

1. **A verified formalization** of the report's resolution lower bounds, Example 4.1, the three Section 5 reductions, and Theorem 4.4 on a timed machine model with honest invertible reductions.

2. **A diagnostic audit** identifying three encoding defects, proving their consequences, and supplying repairs strong enough to restore Theorem 4.4 while excluding the degenerate laws.

Proofs in the text are complete but compact. Where the Lean proof fits in ten lines, the snippet copies it; otherwise the snippet states the theorem and points to the library implementation. Run `./scripts/check_exposition.sh` to verify every snippet independently.

---

3 Verified results

This section states the positive canon. Each theorem carries our own number; §6 maps back to TR1995.

3.1 Theorem 1 (Resolution lower bound; TR1995 Theorem 3.1)

Fix constants `c > 0` and `k ≥ 3`. Consider the random dense `k`-CNF model `K(c·n, n, k)`: sample `c·n` clauses independently and uniformly from the ordinary `k`-clauses on `n` variables, then erase sign information. If the density satisfies `7/10 ≤ c · 2^(-k)`, then for some `ε > 0` the following event has probability tending to `1`:

> the formula is unsatisfiable, and its minimum resolution refutation length is at least `(1 + ε)^n`.

Proof. The density hypothesis makes the union bound `2^n (1 - 2^(-k))^(cn) → 0` work, since `7/10 > ln 2`. Hence unsatisfiability holds with high probability (WHP) by a direct counting argument.

Separately, project each signed CNF to its unsigned hypergraph and apply CS87's local-sparsity machinery. Lemma 1 (local sparsity) holds WHP on random hypergraphs. Lemma 4 lifts this to joint satisfaction of properties `P(a)` and `Q(a,b)` on the projected hypergraph, WHP. For any formula in the WHP intersection of {unsatisfiable} and {`P ∧ Q`}, resolution completeness

supplies a refutation, and CS87 Lemma 5 lower-bounds its length by `(1/4)(e/2)^(a⌊bn⌋/16)`, which eventually exceeds `(1 + ε)^n` for a suitable ε. A squeeze argument on event probabilities completes the proof. □

The combinatorial chain (Lemmas 1–5, projection, resolution completeness) is fully formalized in `AvgCaseMls/Section3/` with no `sorry`.

3.2 Exposition/ResolutionLowerBound.lean

Exposition/ResolutionLowerBound.lean

Lean 4 Certificate

```
/-
Runnable snippet for arxiv.md, displayed with the resolution lower bound.

These are the two headline statements of the report's Section 3, formalizing
the Chvatal-Szemeredi bound and its averaged corollary.  The density
hypothesis `7/10 <= c * 2^(-k)` is what makes the union bound
`2^n (1 - 2^(-k))^(cn) -> 0` work, since `7/10 > ln 2`.

The proofs are the top-level assembly only; the combinatorial content lives in
the Lemma 1 through Lemma 5 chain, which the paper describes in prose and which
`AvgCaseMls.Section3` proves in full with no hypothesis packages.

Checked against AvgCaseMls.Section3.
-/
import AvgCaseMls.Section3

namespace Exposition.ResolutionLowerBound

open AvgCaseMls.Section3

/--
Random dense `k`-CNF needs exponentially long resolution refutations with
probability tending to `1`.  The event asserts both unsatisfiability and the
lower bound `(1 + ?)^n` on minimum refutation length.
-/
theorem theorem_3_1
    (c k : Nat) (hc : 0 < c) (hk : 3 <= k)
    (hdensity : (7 / 10 : ?) <= (c : ?) * (2 : ?) ^ (-(k : Int))) :
    exists ? : ?, 0 < ? /\
      WithHighProbability (denseRandomCNF c k)
        (denseTheorem31Event c k ?) :=
  theorem31_with_explicit_constants c k hc hk hdensity

/--
The averaged form.  Since the event has probability tending to `1` it
eventually has probability at least `1/2`, and resolution complexity is
nonnegative everywhere, so the expectation is at least half the bound.
-/
theorem theorem_3_2
    (c k : Nat) (hc : 0 < c) (hk : 3 <= k)
    (hdensity : (7 / 10 : ?) <= (c : ?) * (2 : ?) ^ (-(k : Int))) :
    exists ? : ?, 0 < ? /\
      IsAsymptoticOmega (averageResolutionComplexity c k)
        (fun r => (1 + ?) ^ r) :=
  average_resolution_omega c k hc hk hdensity
```

```
/-! ## The combinatorial core

CS87 Lemma 5 is the step that produces the exponential bound.  It is a
deterministic implication: a clause family based on a hypergraph with
properties `P(a)` and `Q(a,b)` that admits a refutation at all admits none
shorter than `(1/4)(e/2)^(a?bn?/16)`.  Theorem 3.1 supplies `P` and `Q` with
high probability, and resolution completeness supplies the refutation. -/

example {n m : Nat} {a b : ?} {H : Hypergraph n m} {F : Fin m -> Clause n}
    (hbased : ClauseFamilyBasedOn H F)
    (hP : H.HasPropertyP a) (hQ : H.HasPropertyQ a b)
    (ha : 0 <= a) (hb0 : 0 <= b) (hb1 : b <= 1) (hba : b <= a / 8)
    (hrefutes : exists cs, ResolutionRefutation F cs) :
    (1 / 4 : ?) * (Real.exp 1 / 2) ^ (a * ?b * n?? / 16) <=
      resolutionComplexity F :=
  cs87_lemma5 hbased hP hQ ha hb0 hb1 hba hrefutes

end Exposition.ResolutionLowerBound
```

3.3 Theorem 2 (Average resolution complexity; TR1995 Theorem 3.2)

Under the same hypotheses as Theorem 1, the expected resolution complexity satisfies

$$\mathbb{E}[\text{resolutionComplexity}(F)] = \Omega\big((1+\varepsilon)^n\big).$$

Proof. Take the ε `> 0` from Theorem 1. Once the event of Theorem 1 has probability at least `1/2`, every formula in that event contributes at least `(1 + ε)^n` to the expectation, and resolution complexity is nonnegative everywhere. Hence the expectation is at least `(1/2)(1 + ε)^n` eventually. Reindexing `n = r + k` yields the stated asymptotic bound. □

(See the same snippet as Theorem 1.)

3.4 Theorem 3 (Example 4.1; TR1995 Example 4.1)

Let the **standard law** assign mass `(6/π²) · |x|⁻² · 2^(-|x|)` to each nonempty bitstring `x` (and zero to the empty string). For any ε `> 0`, the shell contributions

$$\text{shell}_n := \sum_{|x|=n} \mu(x) \cdot T^{-1}(|x|^2)/|x|$$

with inverse time scale `T⁻¹(t) = t^(1/(1+ε))` satisfy `∑_n shell_n < ∞`. After normalization by `C = max(∑_n shell_n, 1)`, the Levin average-time condition holds for any decider running in at most `|x|²` steps.

Proof. On the shell of length `n`, the `2^(-n)` factor cancels and the summand is `(6/π²) · n^(-3 + 2/(1+ε))`. The exponent `-3 + 2/(1+ε)` is strictly below `-1` when ε `> 0`, because `2/(1+ε) < 2`. Comparison with the p-series for exponent `s < -1` gives convergence (Mathlib lemma `Real.summable_nat_rpow`).

Convergence does **not** imply the Levin bound $\leq$ `1`; that requires dividing by `C`. The standard law itself sums to `1` via $\sum$ `n⁻² = π²/6`. □

3.5 Exposition/Example41.lean

Exposition/Example41.lean

Lean 4 Certificate

```
/-
Runnable snippet for arxiv.md, displayed with Example 4.1.

The shell exponent is `-3 + 2/(1+?)`, below `-1` when `? > 0`, so the p-series
converges.  Normalization and the full Levin bound are in the library.

Checked against AvgCaseMls.TR1995 and AvgCaseMls.Example41.
-/
import AvgCaseMls.TR1995
import AvgCaseMls.Example41

namespace Exposition.Example41

example (? : ?) : TR1995.example41Exponent ? = -3 + 2 / (1 + ?) := rfl

theorem example41Exponent_lt_neg_one {? : ?} (h? : 0 < ?) :
    TR1995.example41Exponent ? < -1 := by
  have h1 : (0 : ?) < 1 + ? := by linarith
  have h2 : 2 / (1 + ?) < 2 := by
    rw [div_lt_iff_0 h1]; linarith
  rw [TR1995.example41Exponent]
  linarith

theorem example_4_1 {? : ?} (h? : 0 < ?) :
    Summable (TR1995.example41Contribution ?) := by
  have hp : Summable (fun n : Nat => (n : ?) ^ TR1995.example41Exponent ?) :=
    (Real.summable_nat_rpow).2 (example41Exponent_lt_neg_one h?)
  exact hp.mul_left (6 / Real.pi ^ 2)

example {? : ?} (h? : 0 < ?) :
    exists C : ?, 0 < C /\
      Summable (Example41.levinShellSeries ?) /\
      (sum' n : ?, Example41.levinShellSeries ? n / C) <= 1 :=
  Example41.normalized_levin_bound h?

example : Example41.standardDistribution.mass = 1 :=
  Example41.standardDistribution_mass

end Exposition.Example41
```

3.6 Theorem 4 (Honest invertible reductions preserve completeness; TR1995 Theorem 4.4)

Work in the **timed layer**: languages are decided by explicit `Programs` within polynomial bounds; distributional problems pair a language with a `Subprobability` whose rank function is computed in polynomial time; reductions are injective, computed in polynomial time, and satisfy a rank-domination inequality.

If $\mathtt{L}_1$ is NP-average complete, $\mathtt{L}_2 \in \mathtt{NP}$, and $\mathtt{r} : \mathtt{L}_1 \to \mathtt{L}_2$ is an **honest invertible reduction** (injective, polynomial-time computable, polynomial-time inverse on the range, range recognizable in polynomial time, and honest in output length), then $\mathtt{L}_2$ is NP-average complete.

Proof. Given a distributional-NP source `(L_source, μ_source)`, completeness of $\mathtt{L}_1$ yields a polynomial-time rankable law μ on $\mathtt{L}_1$ and an injective distributional reduction from the source to $(\mathtt{L}_1, \mu)$.

Push μ forward along `r.map` to obtain a law on $\mathtt{L}_2$. Mass is preserved; ranks on the image are preserved exactly (`rankFactor = 1`). Rankability of the pushforward is rebuilt by composing:

test membership in `range(r.map)`, apply the inverse, compute the rank — with fuel bounded using the honesty inequality `|x| ≤ honestyBound(|r.map x|)`.

Package this as an injective distributional reduction into $(L_2, r.\text{transport } \mu)$ and compose with the source-to-L_1 reduction. □

3.7 Exposition/Theorem44Strong.lean

Exposition/Theorem44Strong.lean

Lean 4 Certificate

```
/-
Runnable snippet for arxiv.md, displayed with Theorem 6 (TR1995 Theorem 4.4
on the timed layer).

Honest invertible reductions preserve language-level completeness when target
laws are polynomial-time rankable.  The proof pushes the source law forward
along the reduction map; rankability of the pushforward is rebuilt from range
recognition, the polynomial-time inverse, and the honesty bound.

Full proof in ‘AvgCaseMls/Section4.lean‘.
-/
import AvgCaseMls.Section4

namespace Exposition.Theorem44Strong

open AvgCaseMls.Foundation AvgCaseMls.Section4

theorem theorem_4_4 {L_1 L_2 : Set Bitstring}
    (r : HonestInvertibleReduction L_1 L_2)
    (hL_2NP : InNP L_2)
    (hL_1 : IsNPAverageCompleteLanguage L_1) :
    IsNPAverageCompleteLanguage L_2 := by
  refine <hL_2NP, ?_>
  intro source hsource
  obtain <mu, hmuRankable, sourceToL_1> := hL_1.2 source hsource
  exact <r.transport mu, r.transport_rankable hmuRankable,
    <InjectiveDistributionalReduction.trans sourceToL_1.some
      (r.distributionalReduction mu)>>

end Exposition.Theorem44Strong
```

3.8 Theorems 5–7 (SAT reductions; TR1995 Theorems 5.1–5.3)

All three reductions are stated uniformly: **correctness** (satisfiability ↔ target satisfiability), **injectivity**, an explicit **left inverse** (decoder), and an **exact size identity**. This is precisely the hypothesis package Theorem 4 consumes.

Theorem 5 (SAT → MLS-in). A distinguished variable `x` (index `0`) represents an element; propositional variable `i` becomes set variable `i+1`. Literal `v_i` becomes $x \in s(i)$; literal `¬v_i` becomes $x \notin s(i)$. The empty clause encodes as $x \in x$ (foundation-false); the empty CNF as $x \notin x$ (foundation-true, since equality atoms are unavailable in the fragment). Output size: $5 \cdot |\varphi| - 1$ formula nodes.

Theorem 6 (SAT → EMLS). Each variable `i` gets positive set `4i+1`, negative set `4i+2`, and intersection gadget $4i+7 = \text{pos} \cap \text{neg} = \emptyset$. Clause gadgets build unions along literal chains;

the distinguished element is forced into the union of a satisfied clause's literal sets. The bare semantic core is **not** injective (the complement gadget ignores polarity); tagging with a provenance prefix encoding the source bits repairs this via `toEMLS`.

Theorem 7 (SAT → FPILP). Variables are constrained to `{0,1}` by `x_i ≥ 0` and `1 - x_i ≥ 0`. Each clause becomes `∑_{l ∈ c} term(l) ≥ 1` where `v_i` contributes `x_i` and `¬v_i` contributes `1 - x_i`. Constraint count: `2n + |φ|`.

3.9 Exposition/Reductions.lean

Exposition/Reductions.lean

Lean 4 Certificate

```
/-
Runnable snippet for arxiv.md, displayed with the three SAT reductions.

All three are stated in the same four-part form -- correctness, injectivity, an
explicit left inverse, and an exact size identity -- which is precisely the
hypothesis package that the repaired Theorem 4.4 consumes.  Stating them
uniformly is what makes them composable with the transfer theorem.

Note on the second one: the bare semantic core is *not* injective, because the
complement gadget for a literal ignores its polarity.  Tagging the output with
a provenance prefix that encodes the source bits repairs this, which is why
‘toEMLS‘ rather than ‘semanticCore‘ is the reduction of record.

Checked against AvgCaseMls.MLSInReduction, AvgCaseMls.EMLSReduction, and
AvgCaseMls.FPILP.
-/
import AvgCaseMls.MLSInReduction
import AvgCaseMls.EMLSReduction
import AvgCaseMls.FPILP

namespace Exposition.Reductions

open MLS

/-! ## SAT to the membership fragment of MLS

A distinguished variable ‘x‘ plays the role of an element; propositional
variable ‘i‘ becomes the set variable ‘i+1‘.  A positive literal becomes
‘x in s(i)‘, a negative one ‘x notin s(i)‘.  The empty clause becomes ‘x in x‘ and
the empty formula ‘x notin x‘, both decided by foundation, since the fragment has
no equality atoms available. -/

theorem reduction_5_1 :
    (forall ? : SAT.CNF,
        SAT.Satisfiable ? <-> MLSInReduction.MLSSatisfiable (MLSInReduction.toMLS ?))
    /\
    Function.Injective MLSInReduction.toMLS /\
    (forall ? : SAT.CNF, MLSInReduction.fromMLS (MLSInReduction.toMLS ?) = some ?) /\
    (forall ? : SAT.CNF, MLSInReduction.IsMLSIn (MLSInReduction.toMLS ?)) /\
    forall ? : SAT.CNF,
      formulaNodes (MLSInReduction.toMLS ?) + 1 = 5 * SAT.size ? :=
  <MLSInReduction.satisfiable_iff,
   MLSInReduction.toMLS_injective,
   MLSInReduction.fromMLS_toMLS,
   MLSInReduction.toMLS_isMLSIn,
```

```
   MLSInReduction.formulaNodes_toMLS>

/-! ## SAT to EMLS

Each propositional variable ‘i‘ gets a positive set, a negative set, and a
gadget forcing their intersection empty, so no variable is both true and false.
Each clause is turned into a chain of union gadgets accumulating the sets of
its literals, and the distinguished element is forced into that union, so the
clause holds exactly when one of its literals is true. -/

theorem reduction_5_2 :
    (forall ? : SAT.CNF,
        SAT.Satisfiable ? <->
          EMLSReduction.EMLSSatisfiable (EMLSReduction.toEMLS ?)) /\
    Function.Injective EMLSReduction.toEMLS /\
    (forall ? : SAT.CNF, EMLSReduction.fromEMLS (EMLSReduction.toEMLS ?) = some ?) /\
    forall ? : SAT.CNF,
      (EMLSReduction.toEMLS ?).length =
        (EMLSReduction.sourceBits ?).length + 2 +
          3 * EMLSReduction.literalCount ? + ?.length :=
  <EMLSReduction.toEMLS_satisfiable_iff,
   EMLSReduction.toEMLS_injective,
   EMLSReduction.fromEMLS_toEMLS,
   EMLSReduction.toEMLS_length>

/-! ## SAT to feasibility of integer linear programs

Variables are pinned to ‘{0,1}‘ by the pair of inequalities ‘x? >= 0‘ and
‘1 - x? >= 0‘.  A clause becomes ‘sum terms >= 1‘, where a positive literal
contributes ‘x?‘ and a negative one ‘1 - x?‘, so the clause is satisfied
exactly when some term equals ‘1‘. -/

theorem reduction_5_3 :
    (forall {n : Nat} (? : TR1995.FPILPSource.CNF n),
        ?.Satisfiable <-> (TR1995.FPILPSource.satToFPILP ?).Feasible) /\
    (forall {n : Nat}, Function.Injective (@TR1995.FPILPSource.satToFPILP n)) /\
    forall {n : Nat} (? : TR1995.FPILPSource.CNF n),
      (TR1995.FPILPSource.satToFPILP ?).constraints.length = 2 * n + ?.length :=
  <fun ? => (TR1995.FPILPSource.satToFPILP_feasible_iff ?).symm,
   @TR1995.FPILPSource.satToFPILP_injective,
   TR1995.FPILPSource.satToFPILP_constraint_count>

end Exposition.Reductions
```

---

4 What the formalization found

Auditing the repository's initial untimed approximation of the report's RS93-based vocabulary exposes three independent defects. Each admits a concrete degenerate witness; together they trivialize Theorems 4.1, 4.4 (in the untimed layer), and the entire conditional hardness chain.

4.1 3.1 Degenerate laws

Both the untimed `AvCom.Distribution` and the timed `Foundation.Subprobability` bound total mass by `1` rather than fixing it at `1`. Both rank functions return `0` wherever probability

vanishes. The everywhere-zero law is therefore admissible, polynomial-time rankable, and has rank identically zero — making the domination inequality `0 ≤ _` free.

4.2 Exposition/DegenerateLaws.lean

Exposition/DegenerateLaws.lean

Lean 4 Certificate

```
/-
Runnable snippet for arxiv.md.  Displayed alongside the discussion of the two
degenerate laws that drive the collapse results.

Both layers bound total mass by `1` rather than fixing it at `1`, and both
return rank `0` off support.  The everywhere-zero measure is therefore a legal
law whose rank vanishes identically -- which is what makes the rank domination
inequality free.

Checked against AvgCaseMls.AvCom and AvgCaseMls.Section4.
-/
import AvgCaseMls.AvCom
import AvgCaseMls.Section4

namespace Exposition.DegenerateLaws

/-! ## The untimed layer -/

section Untimed
open AvCom

/-- `prob_sum_le_one` bounds total mass by `1`, so this is a legal law. -/
def zeroDistribution : Distribution where
  support := empty
  prob := fun _ => 0
  prob_nonneg := fun _ => le_refl 0
  prob_zero_outside := fun _ _ => rfl
  prob_sum_le_one := by simp

/-- Its rank vanishes everywhere, because `rank` returns `0` off support. -/
@[simp] theorem rank_zeroDistribution (x : Bitstring) :
    rank zeroDistribution x = 0 := by
  simp [rank, zeroDistribution]

/-- And a constant rank function is polynomially rankable. -/
theorem zeroDistribution_polRankable : IsPolRankable zeroDistribution :=
  <fun _ => 0, <0, 0, by simp>, fun x => by simp>

end Untimed

/-! ## The timed layer -/

section Timed
open AvgCaseMls.Foundation

/-- `tsum_le_one` likewise only bounds the mass. -/
noncomputable def zeroLaw : Subprobability where
  prob := fun _ => 0
  summable_prob := summable_zero
```

```
  tsum_le_one := by simp
  finite_superlevel := fun _ hx => absurd rfl hx

@[simp] theorem zeroLaw_prob (x : Bitstring) : zeroLaw.prob x = 0 := rfl

@[simp] theorem zeroLaw_rank (x : Bitstring) : zeroLaw.rank x = 0 :=
  Subprobability.rank_eq_zero_of_prob_eq_zero _ _ rfl

/-- It is not a probability measure; this is the defect the repair fixes. -/
theorem zeroLaw_mass : zeroLaw.mass = 0 := by
  simp [Subprobability.mass]

/-- Rankable in the timed sense too: a one-step constant program computes it. -/
theorem zeroLaw_rankable : IsPolynomialTimeRankable zeroLaw := by
  refine <.constant true (encodeNat 0), fun _ => 1, IsPolynomial.const 1,
    monotone_const, ?_>
  intro x
  exact <<true, encodeNat 0, 1>, rfl, by simp>

end Timed

end Exposition.DegenerateLaws
```

4.3 3.2 Collapse I: membership is free (Theorem 8)

Theorem 8. In the untimed `AvCom` layer, `InNP L` holds for every language `L`.

Proof. `InNP` quantifies over an arbitrary verifier `verify : Bitstring → Bitstring → Bool` and bounds only certificate *length*, never the cost of running `verify`. Take `verify x w := decide(x ∈ L)` with certificate bound zero. The empty certificate witnesses membership. □

4.4 Exposition/InNPTrivial.lean

Exposition/InNPTrivial.lean

Lean 4 Certificate

```
/-
Runnable snippet for arxiv.md, displayed with the proof that the report's
membership condition is vacuous.

‘AvCom.InNP‘ quantifies over an arbitrary function
‘verify : Bitstring -> Bitstring -> Bool‘ and constrains only the length of the
certificate.  Nothing bounds the cost of running ‘verify‘, so the classical
decision procedure for ‘L‘ witnesses membership with the empty certificate.

Checked against AvgCaseMls.AvCom.
-/
import AvgCaseMls.AvCom

namespace Exposition.InNPTrivial

open AvCom

/-- Every language is in the report's ‘NP‘. -/
theorem inNP_trivial (L : Set Bitstring) : InNP L := by
  classical
```

```
  -- The verifier is the characteristic function of ‘L‘; the certificate bound
  -- is the zero polynomial, so certificates must be empty.
  refine <fun x _ => decide (x in L), fun _ => 0, <0, 0, by simp>, ?_>
  intro x
  exact <fun hx => <[], by simp [len], by simpa using hx>,
         fun <_, _, hv> => by simpa using hv>

end Exposition.InNPTrivial
```

Theorem 9 (Characterization of untimed completeness; no TR1995 counterpart). `TR1995.IsNPAverageCompleteLanguage L` if and only if `L` $\neq \emptyset$ and `L` $\neq$ `Bitstring`.

Proof sketch. Forward: reduce from the universal and empty sources using the zero law; nontriviality forces a member and a non-member. Backward: given `a` $\in$ `L` and `b` $\notin$ `L`, map source members to `a` and non-members to `b`; domination is free because target rank is zero. Full proof in the library. □

4.5 Exposition/CompletenessCharacterization.lean

Exposition/CompletenessCharacterization.lean

Lean 4 Certificate

```
/-
Runnable snippet for arxiv.md, displayed with the first collapse theorem.

Language-level NP-average completeness in the report's untimed vocabulary is
equivalent to the language being neither empty nor everything.  The full proof
is in ‘AvgCaseMls/EncodingCollapse.lean‘; it constructs the degenerate law as
target and uses a two-valued reduction map.

Checked against AvgCaseMls.EncodingCollapse.
-/
import AvgCaseMls.EncodingCollapse

namespace Exposition.CompletenessCharacterization

open AvCom

theorem npAverageCompleteLanguage_iff_nontrivial (L : Set Bitstring) :
    TR1995.IsNPAverageCompleteLanguage L <-> (L /= empty /\ L /= Set.univ) :=
  AvgCaseMls.EncodingCollapse.npAverageCompleteLanguage_iff_nontrivial L

end Exposition.CompletenessCharacterization
```

Corollary (TR1995 Theorem 4.4 is empty in this layer). Completeness transfers along any map satisfying the correctness biconditional alone. The injectivity, invertibility, range recognition, forward length, and honesty fields — the hypotheses the report's Theorem 4.4 is about — are never used.

4.6 Exposition/Theorem44Vacuous.lean

Exposition/Theorem44Vacuous.lean

Lean 4 Certificate

```
/-
```

```
Runnable snippet for arxiv.md, displayed with the corollary that the report's
Theorem 4.4 carries no content in its own vocabulary.

The hypotheses of the report's Theorem 4.4 are that the reduction is
injective, polynomial time invertible, and honest.  The proof term below
mentions only `r.map` and `r.reduces`.  Every other field of
`FaithfulReduction` is unused, and so is the `InNP L_2` hypothesis, which
`inNP_trivial` supplies for free.

Checked against AvgCaseMls.EncodingCollapse and AvgCaseMls.HonestReduction.
-/
import AvgCaseMls.EncodingCollapse
import AvgCaseMls.HonestReduction

namespace Exposition.Theorem44Vacuous

open AvCom AvgCaseMls.EncodingCollapse

/-- Completeness transfers along any map satisfying the correctness
biconditional, with no further hypotheses at all. -/
theorem npAverageCompleteLanguage_of_reduces {L_1 L_2 : Set Bitstring}
    (map : Bitstring -> Bitstring) (reduces : forall x, x in L_1 <-> map x in L_2)
    (h_1 : TR1995.IsNPAverageCompleteLanguage L_1) :
    TR1995.IsNPAverageCompleteLanguage L_2 := by
  -- Push a member and a non-member of `L_1` through `map`.
  obtain <hne, hnu> := (npAverageCompleteLanguage_iff_nontrivial L_1).mp h_1
  obtain <a, ha> := Set.nonempty_iff_ne_empty.mpr hne
  obtain <b, hb> := exists_not_mem_of_ne_univ hnu
  exact npAverageCompleteLanguage_of_nontrivial ((reduces a).mp ha)
    (fun h => hb ((reduces b).mpr h))

/-- The report's Theorem 4.4, with its hypotheses visibly unused. -/
theorem theorem_4_4_uses_only_correctness {L_1 L_2 : Set Bitstring}
    (r : HonestReduction.FaithfulReduction L_1 L_2)
    (h_1 : TR1995.IsNPAverageCompleteLanguage L_1) (_ : InNP L_2) :
    TR1995.IsNPAverageCompleteLanguage L_2 :=
  npAverageCompleteLanguage_of_reduces r.map r.reduces h_1

end Exposition.Theorem44Vacuous
```

TR1995 Theorem 4.1 (completeness of a distributional problem implies completeness of its language) is a quantifier shift, not a complexity result; we demote it to a remark in §6.

4.7 3.3 Collapse II: domination is free (Theorem 10)

Moving to the timed layer repairs the verifier and reduction-map gaps: `Foundation.InNP` requires a `Program` deciding within a polynomial bound; injective distributional reductions require a `Program` for the map. But the domination condition still carries no content.

Theorem 10. In the timed layer, language-level NP-average completeness is equivalent to `InNP L` together with injective polynomial-time hardness from every distributional-NP source — with no use of rank domination.

Proof sketch. Forward: forget the three rank fields. Backward: supply `zeroLaw` as the target; discharge domination with rank factor zero. □

4.8 Exposition/DominationFree.lean

Exposition/DominationFree.lean

Lean 4 Certificate

```
/-
Runnable snippet for arxiv.md, displayed with the second collapse theorem.

In the timed model, language-level completeness is equivalent to NP membership
plus injective polynomial-time hardness; the rank domination clause contributes
nothing.  Full proof in ‘AvgCaseMls/DominationCollapse.lean‘.

Checked against AvgCaseMls.DominationCollapse.
-/
import AvgCaseMls.DominationCollapse

namespace Exposition.DominationFree

open AvgCaseMls.Foundation AvgCaseMls.Section4

theorem isNPAverageCompleteLanguage_iff (L : Set Bitstring) :
    IsNPAverageCompleteLanguage L <->
      InNP L /\ forall source : DistributionalProblem, InDistNP source ->
        Nonempty (PolyTimeInjectiveReduction source L) :=
  AvgCaseMls.Section4.isNPAverageCompleteLanguage_iff L

end Exposition.DominationFree
```

4.9 3.4 Collapse III: average tractability and conditional hardness (Theorem 11)

Theorem 11. `AvCom.AvP p` if and only if `IsPolRankable p.`μ. The language `p.L` is irrelevant.

Proof. `AvCom.DistTime T p` reads $\exists$ `f, IsAvTime T f p.`μ. Nothing ties `f` to a decider for `p.L`. Witness `f` $\equiv$ `0`; then `T⁻¹ T 0 = 0` and every rank-truncated sum vanishes. Rankability of the law is the only remaining constraint. □

4.10 Exposition/AvPVacuous.lean

Exposition/AvPVacuous.lean

Lean 4 Certificate

```
/-
Runnable snippet for arxiv.md, displayed with the third collapse theorem.

‘AvCom.DistTime‘ reads

    exists f : Bitstring -> Nat, IsAvTime T f prob.mu

where ‘f‘ is meant to be the running time of a decider for ‘prob.L‘.  Nothing
ties ‘f‘ to any decider, or to ‘prob.L‘ at all, so ‘f ? 0‘ witnesses it.  The
consequence propagates all the way to the conditional hardness results: the
package that states the average-case collapse proves its own collapse, so no
package can assert the separation those results assume.

Checked against AvgCaseMls.ComplexityAxioms.
```

```
-/
import AvgCaseMls.ComplexityAxioms

namespace Exposition.AvPVacuous

open AvCom

/-- The free runtime witness. ‘T_inv T 0 = 0‘, so every truncated sum vanishes. -/
theorem distTime_trivial (T : Nat -> Nat) (p : DistributionalProblem) :
    DistTime T p := by
  refine <fun _ => 0, ?_>
  intro l hl
  have hzero :
      (rankLe p.mu l).sum (fun x => (T_inv T 0 : Real) / (lenBot x : Real)) = 0 := by
    simp [T_inv]
  rw [hzero]
  exact_mod_cast Nat.zero_le l

/-- So average polynomial time degenerates to rankability of the law, a
statement that does not mention the language. -/
theorem avP_iff_polRankable (p : DistributionalProblem) :
    AvP p <-> IsPolRankable p.mu :=
  <fun h => h.1,
   fun h => <h, fun _ => 0, <0, 0, by simp>, distTime_trivial _ _>>

/-- And the inclusion the collapse package characterizes is a theorem. -/
theorem distNP_subseteq_AvP :
    forall p : DistributionalProblem, InDistNP p -> AvP p :=
  fun p hp => (avP_iff_polRankable p).mpr hp.2

/-- Hence any such package proves its own ‘NEXP = EXP‘ ... -/
theorem collapse_forces_NEXP_eq_EXP (theory : AverageCaseCollapseTheory) :
    theory.NEXP_eq_EXP :=
  theory.distNP_subseteq_AvP_iff_NEXP_eq_EXP.mp distNP_subseteq_AvP

/-- ... and none can assert the separation, so every conditional hardness
theorem downstream has an unsatisfiable hypothesis. -/
theorem no_theory_separates (theory : AverageCaseCollapseTheory) :
    not  theory.NEXP_neq_EXP :=
  fun hsep => hsep (collapse_forces_NEXP_eq_EXP theory)

end Exposition.AvPVacuous
```

Corollary. Every distributional-NP problem is in `AvP`, unconditionally. Any `AverageCaseCollapseTheory` package therefore proves `NEXP = EXP` (its characterizing field is `(∀ p, InDistNP p → AvP p) ↔ NEXP = EXP`). No such package satisfies `NEXP ≠ EXP`. Hence every conditional hardness theorem in `AvgCaseMls/NonAvP.lean` — including `SatMLS_average_hard` and `satMLSProb_not_AvP` — has an **unsatisfiable hypothesis**; they are vacuous, not merely conditional.

---

5 Repairs

The collapses isolate encoding defects, not errors in the report's combinatorial or syntactic architecture. Two repairs suffice for the definitions the paper actually uses.

5.1 Repair 1: require `mass = 1` of target laws

Define `Subprobability.IsProbability` μ as μ`.mass = 1`. Require both source and target laws in completeness and reduction statements to be probability measures. This excludes `zeroLaw` and forces some point to have positive rank, making domination a genuine numeric constraint wherever the target charges the image.

Theorem 12 (Theorem 4.4 under the repair). Honest invertible reductions preserve `IsNPAverageCompleteLanguageStrict` — the strengthened definition requiring probability measures. The proof is unchanged in structure: pushforward along an injection preserves total mass.

5.2 Exposition/Repair.lean

Exposition/Repair.lean

Lean 4 Certificate

```
/-
Runnable snippet for arxiv.md, displayed with the two repairs.

Repair 1 requires target laws to be probability measures (‘mass = 1‘).
Repair 2 replaces the free runtime function by an actual decider.

Checked against AvgCaseMls.Repair.
-/
import AvgCaseMls.Repair

namespace Exposition.Repairs

open AvgCaseMls.Foundation AvgCaseMls.Section4 AvgCaseMls.Repair

theorem zeroLaw_not_probability : not  Subprobability.IsProbability zeroLaw := by
  rw [Subprobability.IsProbability, zeroLaw_mass]
  exact zero_ne_one

theorem domination_constrains {source : DistributionalProblem}
    {L : Set Bitstring} {mu : Subprobability}
    (r : InjectiveDistributionalReduction source <L, mu>)
    {x : Bitstring} (h : mu.prob (r.map x) /= 0) :
    1 <= r.rankFactor (len x) * source.distribution.rank x :=
  AvgCaseMls.Repair.domination_constrains r h

theorem theorem_4_4_strict {L_1 L_2 : Set Bitstring}
    (r : HonestInvertibleReduction L_1 L_2)
    (hL_2NP : InNP L_2)
    (hL_1 : IsNPAverageCompleteLanguageStrict L_1) :
    IsNPAverageCompleteLanguageStrict L_2 :=
  AvgCaseMls.Repair.theorem_4_4_strict r hL_2NP hL_1

theorem inAverageP_has_decider {p : DistributionalProblem}
    (h : InAverageP p) : Nonempty (Decider p.language) :=
  AvgCaseMls.Repair.inAverageP_has_decider h

end Exposition.Repairs
```

5.3 Repair 2: tie average time to a decider

Replace the existentially quantified runtime function in `DistTime` by `Foundation.InAverageP`, which quantifies over a genuine `Decider` and measures `actualRuntime`. Unlike `AvP`, this class entails that the language is totally decidable.

Example 4.1 is already stated in this layer (`IsLevinAverageTime` with an explicit `Decider`).

---

6 Remaining open interfaces

Three items from the report are packaged as explicit hypothesis structures, not proved:

Interface	Content	Used for
`LevinNBHData`	Universal reduction from every distNP problem to bounded halting	NBH completeness
`NBHToMLSData`	Cook–Levin compiler NBH $\to$ serialized MLS with domination	SAT-MLS completeness chain
`AverageCaseCollapseTheory`	Collapse equivalence + AvP pullback	Conditional hardness (now known vacuous)

The FOS80 decision procedure `decideMLSSat` is **sound** on a membership-free fragment and **complete** on that same fragment, but **not** complete globally: $(\emptyset = \emptyset) \vee (\emptyset \neq \emptyset)$ is satisfiable yet rejected, because disjunction is outside the conjunctive decoder.

These are honest gaps: the report's prose proofs for the Levin universal construction and the full MLS decision procedure were never formalized here.

---

7 Mapping to TR1995

Our #	Statement	TR1995	Status
1	Resolution lower bound WHP	Thm 3.1	**Proved**
2	Average resolution complexity Ω	Thm 3.2	**Proved**
3	Example 4.1 summability + Levin bound	Ex 4.1	**Proved** (explicit `C`)
4	Honest invertible reductions preserve completeness	Thm 4.4	**Proved** (timed layer)
5	SAT $\to$ MLS-in	Thm 5.1	**Proved**
6	SAT $\to$ EMLS (tagged `toEMLS`)	Thm 5.2	**Proved**
7	SAT $\to$ FPILP	Thm 5.3	**Proved**
8	`InNP` holds for every language	—	**Negative** (Collapse I)

Our #	Statement	TR1995	Status
9	Untimed completeness ↔ nontrivial language	—	**Negative** (Collapse I)
10	Timed completeness ↔ NP + hard, domination free	—	**Negative** (Collapse II)
11	`AvP` ↔ rankability; no theory separates	—	**Negative** (Collapse III)
12	Theorem 4.4 with `mass = 1` laws	Thm 4.4	**Proved** (repair)
—	Completeness of problem ⇒ completeness of language	Thm 4.1	**Remark** (quantifier shift)
—	Levin universal reduction	§4.2–4.3	Open (`LevinNBHData`)
—	NBH → MLS compiler	§5.1, §5.4	Open (`NBHToMLSData`)
—	`decideMLSSat` complete	§7 / FOS80	**Refuted** globally

---

8 Repository and methodology

The Lean development lives at github.com/catskillsresearch/avg_case_mls. As of this writing: **92 modules**, **~27,400 lines**, **~2,100 declarations**, build clean with no `sorry` in the library.

Layer	Lines	Role
`AvgCaseMls/Section3/`	6,833	CS87 resolution lower bounds
`AvgCaseMls/Foundation/`	6,293	Timed machine model, subprobabilities, reductions
`AvgCaseMls/Section4/`	4,783	Theorem 4.4, Example 4.1, Cook–Levin
Collapse + repair	~400	Diagnostic results
Section 5 reductions	~3,000	MLS-in, EMLS, FPILP

Exposition snippets. The `Exposition/` directory holds runnable versions of every proof displayed in this document. Each file restates the theorem in context and either copies the proof (when it fits in ten lines) or applies it from the library. Check all snippets:

```
./scripts/check_exposition.sh
```

Palomar. `Challenge.lean` remains the comparator surface for external verification; Palomar size caps are deferred until the canon is complete.

---

9 References

- **[Ajt96]** Ajtai, M. (1996). Generating hard instances of lattice problems. *STOC*.

- **[BDCGL89]** Ben-David, S., Chor, B., Goldreich, O., & Luby, M. (1989). On the theory of average case complexity. *STOC*.
- **[CEM95]** Cox, J., Ericson, L., & Mishra, B. (1995). The average case complexity of multilevel syllogistic. *NYU Courant Institute Technical Report TR1995-711*.
- **[COPE24]** Committee on Publication Ethics (COPE). (2024). Authorship and AI tools: COPE position statement. https://publicationethics.org/guidance/cope-position/authorship-and-ai-tools
- **[Cur25]** Anysphere, Inc. Cursor: AI-native code editor and agent environment. https://cursor.com (accessed 2025).
- **[deM08]** de Moura, L., & Bjørner, N. (2008). Z3: An efficient SMT solver. *TACAS*.
- **[DS77]** Davis, M., & Schwartz, J. T. (1977). Metamathematical extensibility for theorem verifiers. *NYU Technical Report*.
- **[FOS80]** Ferro, A., Omodeo, E. G., & Schwartz, J. T. (1980). Decision procedures for elementary sublanguages of set theory. *CPAM*.
- **[Gol79]** Goldberg, A. T. (1979). On the complexity of the satisfiability problem. *NYU PhD Thesis*.
- **[Gur91]** Gurevich, Y. (1991). Average case completeness. *Journal of Computer and System Sciences*.
- **[Lev86]** Levin, L. (1986). Average case complete problems. *SIAM Journal on Computing*.
- **[RS93]** Reischuk, R., & Schindelhauer, C. (1993). Precise average case complexity. *STOC*.
- **[Ste23a]** Stevens, L. (2023). MLSS Decision Procedure. *Archive of Formal Proofs*. https://isa-afp.org/entries/MLSS_Decision_Proc.html
- **[Ste23b]** Stevens, L. (2023). Towards a Verified Tableau Prover for a Quantifier-Free Fragment of Set Theory. In Pientka, B., & Tinelli, C. (eds.), *Automated Deduction – CADE 29*, LNAI 14132, 491–508. https://doi.org/10.1007/978-3-031-38499-8_28
- **[SY92]** Schnorr, C. P., & Yoshida, T. (1992). Average-case complexity of NP-complete problems. *STOC*.
- **[Sny90a]** Snyder, W. K. (1990). The SETL2 programming language. *NYU Technical Report*.
- **[ST01]** Spielman, D. A., & Teng, S. H. (2001). Smoothed analysis of algorithms. *STOC*.
- **[VR92]** Venkatesan, R., & Rajagopalan, S. (1992). Average case intractability of matrix and Diophantine problems. *STOC*.